

Remote Desktop Protocols in Linux

X11 · VNC (TigerVNC) · RDP (xrdp)

Cybersecurity Home Lab — Personal Study Notes

[Ihar Shcharbitski] · [18.05.2026]

Machine	Role
Parrot Security OS 7.4 (admin workstation)	CLIENT. Runs VNC viewer (tigervnc-viewer) and RDP client (freerdp3-x11). SSH-tunnels into Ubuntu. Displays and controls Ubuntu's desktop. This is where I sit.
Ubuntu 26.04 (remote server — second VM)	SERVER. Runs TigerVNC server, XFCE4 desktop, and xrdp. Exposes its desktop via VNC (port 5902) and RDP (port 3389). Configured remotely; never accessed at the console.

1. Lab Overview

This lab replicates a real-world scenario in which a security engineer remotely manages a headless Linux server using graphical desktop protocols. Ubuntu acts as the remote server — it runs a full XFCE4 desktop session that is accessed over the network from Parrot OS, the admin workstation. Three protocols are practiced in order of increasing complexity:

- X11 Forwarding — single-application remote display; no additional software required beyond SSH.
- VNC over SSH Tunnel — full remote desktop session encrypted inside an SSH tunnel; the industry-standard secure approach for Linux-to-Linux access.
- RDP via xrdp — Microsoft's Remote Desktop Protocol implemented on Linux; dominant in enterprise Windows-to-Linux access scenarios.

2. Environment Setup

2.1 Find IP Addresses

An IP address is required for each machine to establish network connections. Run `ip a` on each machine and note the address on the Host-Only adapter (e.g. 192.168.56.x). Use these addresses in all subsequent steps.

→ `ip a` (Parrot OS)

```
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:d6:06:ad brd ff:ff:ff:ff:ff:ff
    altname enx080027d606ad
    inet 192.168.56.103/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s8
        valid_lft 436sec preferred_lft 436sec
    inet6 fe80::e1c4:31b3:3927:68eb/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Fig 2.1 — Network interface addresses on Parrot OS (admin workstation)

→ `ip a` (Ubuntu)

```
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:49:95:f1 brd ff:ff:ff:ff:ff:ff
    altname enx0800274995f1
    inet 192.168.56.104/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s8
        valid_lft 475sec preferred_lft 475sec
    inet6 fe80::b19:d952:b216:3746/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Fig 2.2 — Network interface addresses on Ubuntu (remote server)

2.2 Install and Start OpenSSH Server on Ubuntu

The SSH service is the secure transport layer that underpins all remote access in this lab. VNC traffic will travel encrypted inside an SSH tunnel; X11 forwarding is also carried by SSH.

→ `sudo apt update && sudo apt install openssh-server -y`

```
shcherba@Ubuntu-26-04:~$ sudo apt install openssh-server
```

Fig 2.3 — Installing OpenSSH server on Ubuntu

→ `sudo systemctl start ssh && sudo systemctl status ssh`

```

shcherba@Ubuntu-26-04:~$ sudo systemctl start ssh
shcherba@Ubuntu-26-04:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: active (running) since Sun 2026-05-17 20:04:29 UTC; 12s ago
     Invocation: 4aadb7d28df154222bc8a7b96754c2045
   TriggeredBy: ● ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
    Process: 5952 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 5955 (sshd)
      Tasks: 1 (limit: 2804)
     Memory: 1.2M (peak: 2M)
        CPU: 18ms
     CGroup: /system.slice/ssh.service
            └─5955 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

May 17 20:04:29 Ubuntu-26-04 systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
May 17 20:04:29 Ubuntu-26-04 sshd[5955]: Server listening on 0.0.0.0 port 22.
May 17 20:04:29 Ubuntu-26-04 sshd[5955]: Server listening on :: port 22.
May 17 20:04:29 Ubuntu-26-04 systemd[1]: Started ssh.service - OpenBSD Secure Shell server.

```

Fig 2.4 — Enabling, starting, and verifying the SSH service on Ubuntu

2.3 Verify SSH Connectivity from Parrot

Before configuring any remote desktop protocol, confirm that Parrot can successfully authenticate to Ubuntu over SSH. This validates network reachability and credential correctness.

→ `ssh <username>@<UBUNTU_IP>`

```

[user@parrot]~$ ssh shcherba@192.168.56.104
The authenticity of host '192.168.56.104 (192.168.56.104)' can't be established.
ED25519 key fingerprint is SHA256:2aAT8BBV0D+V90Gf2GhYV5/Eb+LLm5Jkh1j+wfwBHL0.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '192.168.56.104' (ED25519) to the list of known hosts.
shcherba@192.168.56.104's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-15-generic x86_64)

```

Fig 2.5 — Test SSH login from Parrot to Ubuntu

2.4 Install XFCE4 Desktop Environment on Ubuntu

XFCE4 is selected over GNOME or KDE because it is lightweight and compositor-free, making it highly stable when rendered over a VNC framebuffer. Heavier desktop environments are prone to rendering artefacts and crashes in virtual display sessions.

→ `sudo apt install xfce4 xfce4-goodies -y`

```

shcherba@Ubuntu-26-04:~$ sudo apt install xfce4 xfce4-goodies -y
The following packages were automatically installed and are no longer required:
  linux-image-unsigned-7.0.0-14-generic linux-tools-7.0.0-14
  linux-main-modules-zfs-7.0.0-14-generic linux-tools-7.0.0-14-generic
  linux-modules-7.0.0-14-generic
Use 'sudo apt autoremove' to remove them.

Installing:
  xfce4 xfce4-goodies

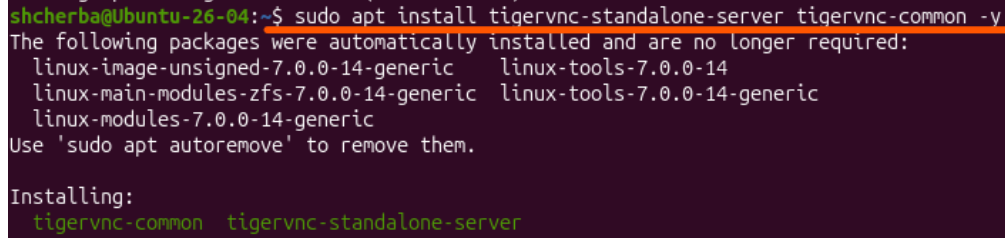
```

Fig 2.6 — Installing XFCE4 and companion utilities on Ubuntu

2.5 Install TigerVNC Server on Ubuntu

TigerVNC is the industry-standard open-source VNC server for Linux. The standalone-server package creates self-contained virtual display sessions that do not require a physical monitor to be attached to Ubuntu.

```
→ sudo apt install tigervnc-standalone-server tigervnc-common -y
```



```
shcherba@Ubuntu-26-04:~$ sudo apt install tigervnc-standalone-server tigervnc-common -y
The following packages were automatically installed and are no longer required:
  linux-image-unsigned-7.0.0-14-generic  linux-tools-7.0.0-14
  linux-main-modules-zfs-7.0.0-14-generic  linux-tools-7.0.0-14-generic
  linux-modules-7.0.0-14-generic
Use 'sudo apt autoremove' to remove them.

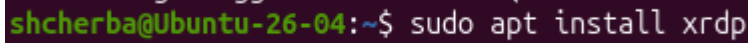
Installing:
  tigervnc-common  tigervnc-standalone-server
```

Fig 2.7 — Installing TigerVNC server packages on Ubuntu

2.6 Install xrdp on Ubuntu

xrdp implements Microsoft's Remote Desktop Protocol on Linux. Once configured, any standard RDP client — including the native Windows Remote Desktop client and xfreerdp on Linux — can connect to Ubuntu's desktop.

```
→ sudo apt install xrdp -y
```



```
shcherba@Ubuntu-26-04:~$ sudo apt install xrdp
```

Fig 2.8 — Installing xrdp on Ubuntu

2.7 Verify All Server Packages on Ubuntu

Confirm that all three server-side packages are installed before proceeding. The dpkg command queries the package database; grep -E enables multi-pattern matching with the | (OR) operator.

```
→ dpkg -l | grep -E 'tigervnc|xrdp|xfce4'
```



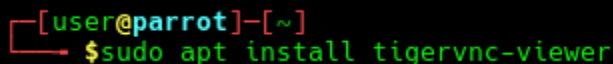
```
shcherba@Ubuntu-26-04:~$ dpkg -l | grep -E "tigervnc|xfce4|xrdp"
ii  libxfce4panel-2.0-4                4.20.7-1
    fce4 panel library (GTK3 variant)
ii  libxfce4ui-2-0:amd64              4.20.2-1
    idget library for Xfce - Gtk+3 variant
ii  libxfce4ui-common                 4.20.2-1
    ommon files for libxfce4ui
ii  libxfce4ui-utils                  4.20.2-1
    tility files for libxfce4ui
ii  libxfce4util-bin                  4.20.1-1build1
    ools for libxfce4util
```

Fig 2.9 — All server packages confirmed installed on Ubuntu (couldn't place everything in one screenshot)

2.8 Install Client Tools on Parrot

Two client-side tools are required on Parrot: a VNC viewer to display Ubuntu's virtual desktop, and an RDP client to connect via the RDP protocol in Section 6.

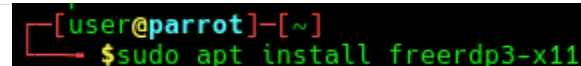
```
→ sudo apt install tigervnc-viewer -y
```



```
[user@parrot]~$ sudo apt install tigervnc-viewer
```

Fig 2.10 — Installing TigerVNC viewer on Parrot

```
→ sudo apt install freerdp3-x11 -y
```



```
[user@parrot]~$ sudo apt install freerdp3-x11
```

Fig 2.11 — Installing FreeRDP client on Parrot

2.9 Verify Client Tools on Parrot

The which command confirms that both binaries are available in PATH. If either returns no output, reinstall the corresponding package.

→ `which vncviewer && which xfreerdp`

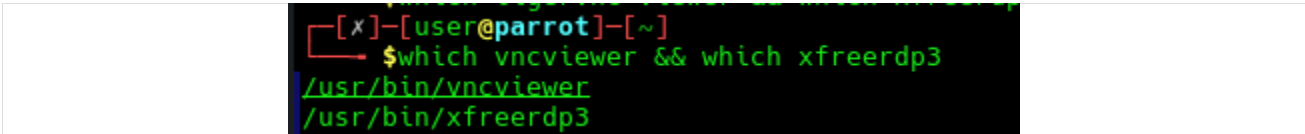


Fig 2.12 — VNC viewer and RDP client confirmed on Parrot

3. X11 Forwarding

X11 Forwarding is the most primitive form of remote display in Linux. Unlike VNC or RDP — which stream an entire desktop — X11 Forwarding relays the drawing instructions of a single application from a remote machine (Ubuntu) to the local X display server (Parrot), where the window appears on screen.

CONCEPT The X Window System (X11)

X11 is the display server architecture that draws windows on screen in Linux. The X server controls the physical monitor and input devices — it runs on Parrot, the local machine (my case). Remote applications are X clients: they send drawing commands to the X server across the network. X11 was designed for this network transparency from its inception. VNC exploits the same principle at a higher level, capturing the full desktop framebuffer rather than individual drawing commands.

CONCEPT X11 vs VNC — Efficiency Trade-off

X11 Forwarding transmits only drawing instructions, not pixels, making it bandwidth-efficient for simple applications. However, modern applications increasingly bypass X11 with direct GPU rendering (OpenGL, Vulkan), which cannot be forwarded. VNC transmits raw framebuffer pixels (or compressed equivalents), which works for any application regardless of how it renders internally.

3.1 Enable X11 Forwarding in sshd_config

X11 Forwarding must be explicitly permitted in Ubuntu's SSH server configuration. The default configuration may have it disabled.

→ `sudo vim /etc/ssh/sshd_config`

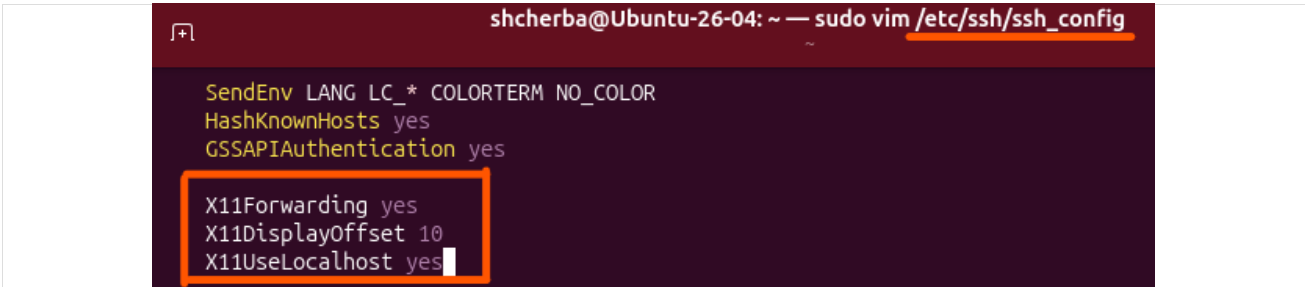


Fig 3.1 — X11 forwarding options in /etc/ssh/sshd_config on Ubuntu

Option	Purpose
X11Forwarding yes	Permits SSH to tunnel X11 display traffic. Without this directive, the -X flag in the client command is silently ignored.

X11DisplayOffset 10	Forwarded X displays are numbered from :10 onwards, preventing collisions with any existing local display sessions.
X11UseLocalhost yes	Binds the X11 forwarding listener to the loopback interface on Ubuntu. X11 traffic does not appear on the network interface; the SSH tunnel is the only delivery path.

3.2 Restart SSH Service

Configuration changes to `sshd_config` take effect only after the SSH service is restarted.

→ `sudo systemctl restart ssh`

```
shcherba@Ubuntu-26-04:~$ sudo systemctl restart ssh
shcherba@Ubuntu-26-04:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; disabled; preset: enabled)
   Active: active (running) since Sun 2026-05-17 20:40:41 UTC; 12s ago
     Invocation: 320debde53034b12a808cab19b54c679
   TriggeredBy: ● ssh.socket
     Docs: man:sshd(8)
           man:sshd_config(5)
    Process: 9976 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 9978 (sshd)
      Tasks: 1 (limit: 2804)
     Memory: 1.2M (peak: 2M)
        CPU: 19ms
     CGroup: /system.slice/ssh.service
            └─9978 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

May 17 20:40:41 Ubuntu-26-04 systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
May 17 20:40:41 Ubuntu-26-04 sshd[9978]: Server listening on 0.0.0.0 port 22.
May 17 20:40:41 Ubuntu-26-04 sshd[9978]: Server listening on :: port 22.
May 17 20:40:41 Ubuntu-26-04 systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
```

Fig 3.2 — Restarting the SSH service to apply configuration

3.3 Establish SSH Connection with X11 Forwarding

The `-X` flag instructs the SSH client to negotiate X11 forwarding with the server. Once the session is established, any GUI application launched inside this shell will have its window forwarded to and displayed by Parrot's X server.

→ `ssh -X <username>@<UBUNTU_IP>`

```
[user@parrot]~$ ssh -X shcherba@192.168.56.104
shcherba@192.168.56.104's password:
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-15-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

1 update can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

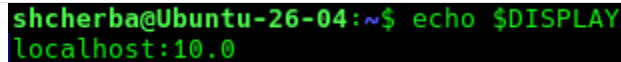
Last login: Sun May 17 20:06:54 2026 from 192.168.56.103
/usr/bin/xauth: file /home/shcherba/.Xauthority does not exist
shcherba@Ubuntu-26-04:~$
```

Fig 3.3 — SSH session opened with X11 forwarding from Parrot to Ubuntu

3.4 Verify the Forwarded Display

When X11 forwarding is active, SSH sets the `DISPLAY` environment variable inside the remote session to point at the forwarded virtual display (e.g. `localhost:10.0`). This is the address of the X11 listener that routes drawing commands back through the tunnel to Parrot's screen.

→ `echo $DISPLAY`



```
shcherba@Ubuntu-26-04:~$ echo $DISPLAY
localhost:10.0
```

Fig 3.4 — `DISPLAY` variable showing the forwarded X11 display address

If the `DISPLAY` variable is empty, X11 forwarding is not functioning. Verify that `X11Forwarding` yes is correctly set in `sshd_config` on Ubuntu, that the service has been restarted, and that the SSH client is connecting with the `-X` flag.

X11 Forwarding is limited to single-application forwarding — one window at a time, with no remote taskbar or window manager. It also carries latency penalties for graphically complex applications. Type `exit` to close the SSH session before continuing to Section 4.

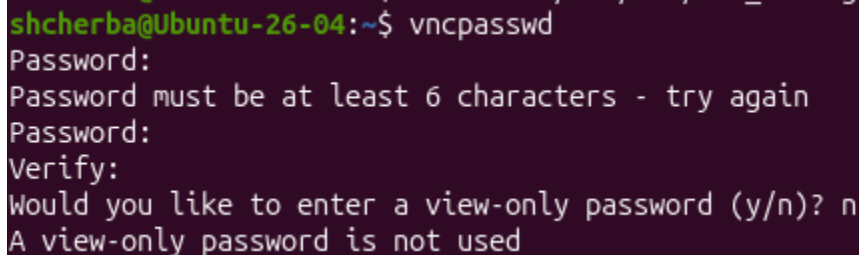
4. TigerVNC Server Configuration on Ubuntu

This section establishes a full virtual desktop session on Ubuntu using TigerVNC. Two configuration files are critical to a functional session: `xstartup`, which defines the desktop environment to launch, and `config`, which controls display parameters and security binding.

4.1 Set the VNC Password

All TigerVNC connections require password authentication. The `vncpasswd` utility creates an obfuscated password file that the VNC server reads at session startup.

→ `vncpasswd`



```
shcherba@Ubuntu-26-04:~$ vncpasswd
Password:
Password must be at least 6 characters - try again
Password:
Verify:
Would you like to enter a view-only password (y/n)? n
A view-only password is not used
```

Fig 4.1 — Setting the VNC connection password with `vncpasswd` on Ubuntu

TigerVNC silently truncates passwords to 8 characters. A password of `'password123'` is stored and accepted as `'password'`. For predictable behaviour, use exactly 8 characters. Note: 8-character passwords are considered weak against modern brute-force attacks; enforce additional authentication layers in production environments.

4.2 Verify the Password File

`vncpasswd` writes its output to `~/.config/tigervnc/passwd`. Confirm the file exists before proceeding.

→ `ls ~/.config/tigervnc/`


```
shcherba@Ubuntu-26-04:~/.config/tigervnc$ ls -la
total 16
drwxrwxr-x  2 shcherba shcherba 4096 May 17 21:13 .
drwx----- 16 shcherba shcherba 4096 May 17 21:06 ..
-rw-rw-r--  1 shcherba shcherba 1004 May 17 21:13 Ubuntu-26-04:2.log
-rw-----  1 shcherba shcherba   8 May 17 21:06 passwd
shcherba@Ubuntu-26-04:~/.config/tigervnc$ cat passwd
I@^" shcherba@Ubuntu-26-04:~/.config/tigervnc$
```

Fig 4.2 — Verifying the VNC password file in ~/.config/tigervnc/

4.3 Create and Configure xstartup

xstartup is a shell script that TigerVNC executes at the beginning of every new desktop session. It is responsible for launching the desktop environment inside the virtual display. An absent, non-executable, or incorrectly written xstartup results in a blank or grey screen on connection.

→ `vim ~/.config/tigervnc/xstartup`

```
shcherba@Ubuntu-26-04: ~/.config/tigervnc — sudo vim xstartup
~/.config/tigervnc

#!/bin/sh

unset SESSION_MANAGER
unset DBUS_SESSION_BUS_ADDRESS
exec startxfce4
```

Fig 4.3 — xstartup file content in the TigerVNC configuration directory

Line	Purpose
<code>#!/bin/sh</code>	Shebang directive. Instructs the OS to execute the script using /bin/sh. Without this line, the OS cannot determine the interpreter and the session does not start.
<code>unset SESSION_MANAGER</code>	Clears any inherited D-Bus session manager reference from a previous graphical session. A stale reference causes XFCE4 to attempt reconnection to a non-existent session and hang at startup.
<code>unset DBUS_SESSION_BUS_ADDRESS</code>	Clears the D-Bus IPC bus address. A stale address causes XFCE4 components (file manager, panel, settings daemon) to fail when they attempt to communicate at session start.
<code>exec startxfce4</code>	Replaces the current shell process with the XFCE4 session manager (exec replaces rather than spawns a child). startxfce4 becomes PID 1 of the session; when the VNC connection closes, the session terminates cleanly without leaving zombie processes.

→ `chmod +x ~/.config/tigervnc/xstartup`

```
shcherba@Ubuntu-26-04:~/.config/tigervnc$ sudo chmod +x xstartup
```

Fig 4.4 — Making the xstartup script executable

TigerVNC refuses to execute a non-executable xstartup file. If `chmod +x` is omitted, the VNC server starts but the connected session displays a blank grey screen. This is the most common beginner error in VNC configuration.

4.4 Verify xstartup Permissions

Confirm that the script is executable before starting the VNC server. The `ls -la` output must show `rwx` bits for the owner (e.g. `-rwxr-xr-x`).

→ `ls -la ~/.config/tigervnc/xstartup`

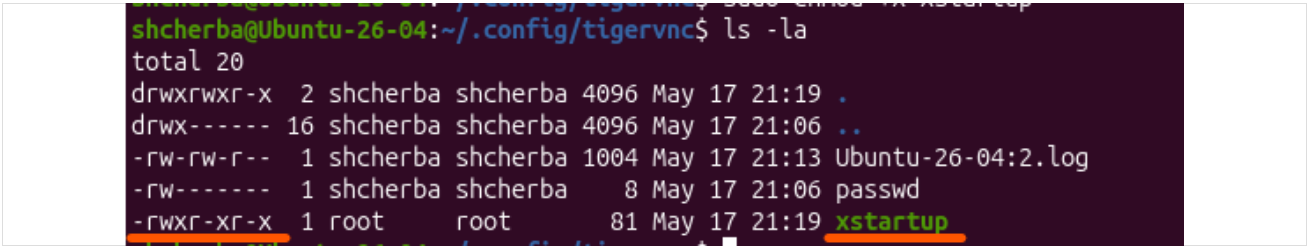


Fig 4.5 — xstartup file permissions — executable bits confirmed

4.5 Create and Configure the config File

The config file controls the server-side display parameters. Settings apply at session start and cannot be changed for a running session without killing and restarting the VNC server.

→ `vim ~/.config/tigervnc/config`

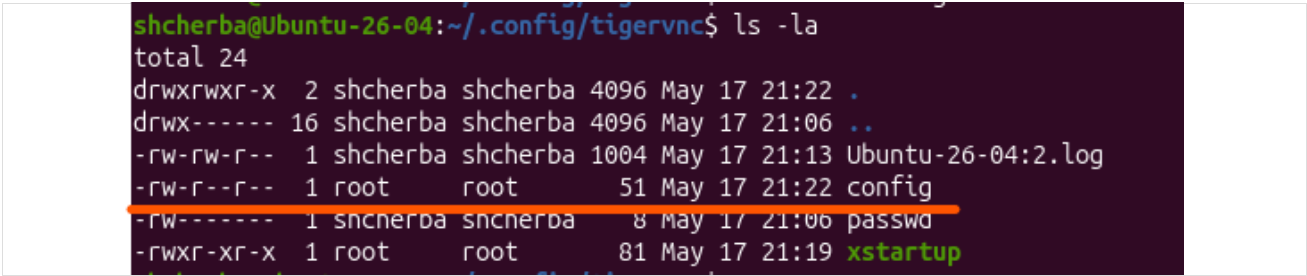


Fig 4.6 — config file content in the TigerVNC configuration directory

Option	Purpose
geometry=1280x720	Sets the virtual framebuffer resolution. 1280x720 is a practical lab default: large enough for usability, small enough to minimise network overhead. Alternatives: 1920x1080, 1440x900.
depth=24	Colour depth in bits per pixel. 24-bit provides full true colour (16.7 million colours). Lower values (16, 8) reduce bandwidth at the cost of colour fidelity.
localhost	CRITICAL: Binds TigerVNC to the loopback interface (127.0.0.1) only. Port 5902 is not reachable from the network. Access is only possible through an SSH tunnel, which provides the encrypted transport path.
alwaysshared	Allows multiple VNC clients to connect to the same session simultaneously. Acceptable in a lab environment; remove in production to enforce exclusive session ownership.

4.6 Start the TigerVNC Server

The `vncserver` command launches a new virtual display session. Display number `:2` maps to port 5902 (formula: `port = 5900 + display_number`). TigerVNC reads the `config` and `xstartup` files and starts an XFCE4 session inside the virtual framebuffer.

→ `vncserver :2`

```
shcherba@Ubuntu-26-04:~/.config/tigervnc$ vncserver :2
New Xtigervnc server 'Ubuntu-26-04:2 (shcherba)' on port 5902 for display :2.
Use xtigervncviewer -SecurityTypes VncAuth -passwd /home/shcherba/.config/tigervnc/passwd :2 to connect
to the VNC server.
```

Fig 4.7 — Starting TigerVNC server on display :2 (port 5902)

To stop a running session: `vncserver -kill :2`. If a session is already running on :2 when attempt to start, kill it first, then start fresh. To restart after config changes, always kill then start — do not attempt to reload in place.

4.7 List Active VNC Sessions

Verify that the session is running on the expected display and port before attempting a connection.

→ `vncserver -list`

```
TigerVNC server sessions:
X DISPLAY #      RFB PORT #      RFB UNIX PATH    PROCESS ID #     SERVER
2              5902              13225            Xtigervnc
```

Fig 4.8 — Active VNC sessions listed — display :2 confirmed on port 5902

4.8 Verify Loopback Binding

The localhost directive in the config file must result in TigerVNC binding exclusively to 127.0.0.1. Confirm this with `ss` before establishing the SSH tunnel. If the process is listening on 0.0.0.0:5902, the config was not applied correctly — kill the session, correct the config file, and restart.

→ `ss -tlnp | grep 5902`

```
shcherba@Ubuntu-26-04:~/.config/tigervnc$ ss -tlnp | grep "5902"
LISTEN 0      5          127.0.0.1:5902  0.0.0.0:*    users:(("Xtigervnc",pid=13225,fd=16))
LISTEN 0      5          [:::1]:5902    [:::]*       users:(("Xtigervnc",pid=13225,fd=17))
```

Fig 4.9 — VNC bound to 127.0.0.1 only — not reachable from the network directly

5. Secure VNC Connection via SSH Tunnel

Raw VNC transmits all traffic — keystrokes, screen contents, and the authentication password — in plaintext over the network. Any host in a position to intercept the traffic can capture the complete desktop session in real time. The professional solution is local port forwarding: routing VNC traffic inside an established SSH tunnel.

An SSH tunnel creates an encrypted channel between two machines. Local port forwarding additionally instructs the SSH process to listen on a specified port on the local machine and forward any traffic received on that port through the encrypted channel to a designated host and port on the remote machine. In this lab: the ssh process on Parrot listens on localhost:5901. `vncviewer` connects to Parrot's localhost:5901. SSH intercepts the traffic, encrypts it, and delivers it to Ubuntu's localhost:5902 (where TigerVNC is listening). The VNC client and VNC server never communicate directly; each communicates only with its local SSH process.

5.1 Create the SSH Tunnel from Parrot

→ `ssh -L 5901:localhost:5902 -N -f <username>@<UBUNTU_IP>`

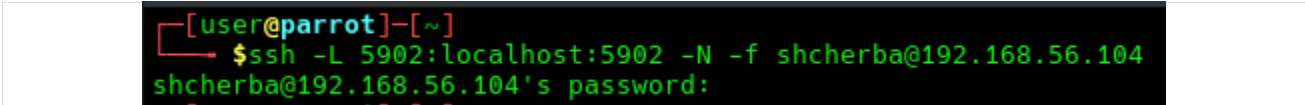


Fig 5.1 — Creating the SSH local port forwarding tunnel from Parrot to Ubuntu

Flag	Purpose
-L 5902:localhost:5902	Local port forwarding. Listen on Parrot's port 5901. Forward all received traffic through the SSH connection to localhost:5902 on Ubuntu — where TigerVNC is bound.
-N	Do not execute a remote command. The SSH session exists solely to maintain the tunnel; no interactive shell is opened on Ubuntu.
-f	Fork into the background after authentication. The tunnel runs as a background process, leaving the terminal available for other commands.
username@UBUNTU_IP	SSH authentication target. The tunnel is created through the authenticated SSH session to the specified user on Ubuntu.

5.2 Verify the Tunnel on Parrot

After the tunnel is created, the SSH process holds Parrot's port 5901 open on the loopback interface. Verify this before connecting the VNC viewer.

→ `ss -tlnp | grep 5902`

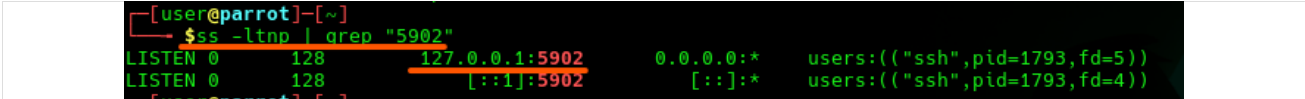


Fig 5.2 — SSH process holding port 5902 on Parrot's loopback — tunnel confirmed

5.3 Connect VNC Viewer Through the Tunnel

The VNC viewer connects to Parrot's localhost:5902 — the local end of the SSH tunnel. It has no direct knowledge of Ubuntu. SSH intercepts the VNC connection, encrypts the data, and delivers it to TigerVNC on Ubuntu's port 5902. Enter the VNC password set in Step 4.1 when prompted.

→ `vncviewer localhost:5901`

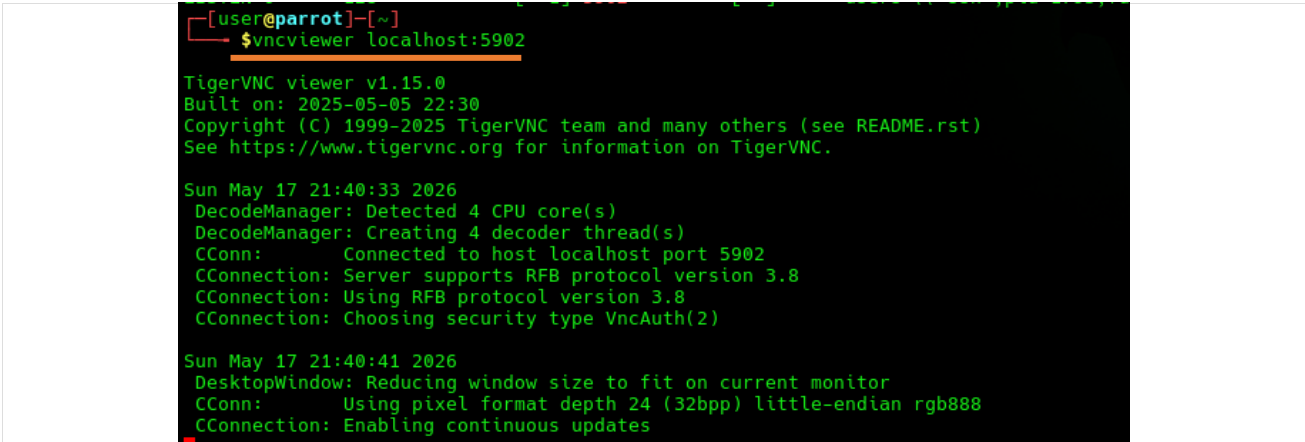


Fig 5.3 — VNC viewer connected through the SSH tunnel — Ubuntu XFCE4 desktop on screen

vncviewer (Parrot) connects to localhost:5902 → SSH process (Parrot) intercepts, encrypts, sends through port 22 → sshd (Ubuntu) decrypts, forwards to localhost:5902 → TigerVNC processes VNC protocol, renders XFCE4 framebuffer. A network sniffer between the two machines observes only encrypted SSH traffic on port 22. No VNC data is exposed on the network.

5.4 Terminate the VNC Session

When the session is no longer required, kill it cleanly. Restarting the server (kill then start) is the correct method for applying any configuration changes made to xstartup or config.

→ `vncserver:2`

```
shcherba@Ubuntu-26-04:~$ vncserver :2
```

```
The X session exited with status 1!
Killing Xtigervnc process ID 5513... success!
```

The tunnel process on Parrot can be terminated by identifying and killing the background ssh process: `kill $(pgrep -f 'ssh -L 5902')`.

6. RDP Connection via xrdp

RDP (Remote Desktop Protocol) is Microsoft's remote desktop standard and the dominant protocol in enterprise Windows-to-Linux access scenarios. xrdp is an open-source implementation of the RDP server for Linux. Unlike VNC, RDP incorporates TLS encryption natively — a direct connection is encrypted without requiring an SSH tunnel.

	VNC (TigerVNC)	RDP (xrdp)
Protocol origin	Open standard — cross-platform	Microsoft — dominant in enterprise
Encryption	None by default; requires SSH tunnel	Built-in TLS — encrypted by default
Session model	Shares a virtual or existing session	Always creates a new independent session
Transport	Raw framebuffer pixels (compressed)	Drawing commands — more bandwidth-efficient
Typical use	Linux-to-Linux admin, headless servers	Windows/Linux-to-Linux corporate access, cloud VMs
Default port	5900 + display number	3389 (fixed)

6.1 Start xrdp

xrdp may have started automatically after installation. Can enable it for persistence across reboots, but for now will just start the service.

→ `sudo systemctl start xrdp`

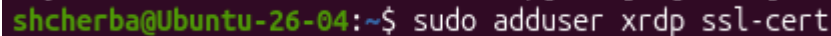
```
shcherba@Ubuntu-26-04:~$ sudo systemctl start xrdp
```

Fig 6.1 —Starting xrdp service on Ubuntu

6.2 Add xrdp to the ssl-cert Group

The xrdp service account requires read access to TLS certificate files to negotiate encrypted RDP sessions. Adding it to the ssl-cert group grants that access. A service restart is required for the group membership to take effect.

→ `sudo adduser xrdp ssl-cert`



```
shcherba@Ubuntu-26-04:~$ sudo adduser xrdp ssl-cert
```

Fig 6.2 — Adding the xrdp service account to the ssl-cert group

6.3 Restart and Verify xrdp

→ `sudo systemctl restart xrdp && sudo systemctl status xrdp`



```
shcherba@Ubuntu-26-04:~$ sudo systemctl restart xrdp
shcherba@Ubuntu-26-04:~$ sudo systemctl status xrdp
● xrdp.service - xrdp daemon
   Loaded: loaded (/usr/lib/systemd/system/xrdp.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-05-17 22:06:03 UTC; 22s ago
     Invocation: 12ea7e5eb2044c31a0b28b01ee2a85b8
       Docs: man:xrdp(8)
            man:xrdp.ini(5)
    Main PID: 14601 (xrdp)
      Tasks: 1 (limit: 2804)
     Memory: 1M (peak: 2.2M)
        CPU: 12ms
    CGroup: /system.slice/xrdp.service
            └─14601 /usr/sbin/xrdp --nodaemon

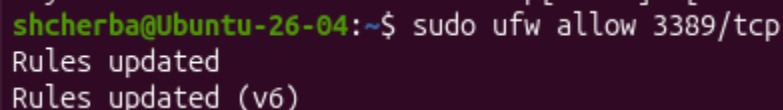
May 17 22:06:03 Ubuntu-26-04 systemd[1]: Starting xrdp.service - xrdp daemon...
May 17 22:06:03 Ubuntu-26-04 systemd[1]: Started xrdp.service - xrdp daemon.
May 17 22:06:03 Ubuntu-26-04 xrdp[14601]: [INFO ] starting xrdp with pid 14601
May 17 22:06:03 Ubuntu-26-04 xrdp[14601]: [INFO ] address [0.0.0.0] port [3389] mode 1
May 17 22:06:03 Ubuntu-26-04 xrdp[14601]: [INFO ] listening to port 3389 on 0.0.0.0
May 17 22:06:03 Ubuntu-26-04 xrdp[14601]: [INFO ] xrdp_listen_pp done
```

Fig 6.3 — xrdp service restarted and confirmed active after ssl-cert group change

6.4 Open Port 3389 in the Firewall

Unlike the VNC setup, where all traffic is routed through the SSH tunnel on port 22, this lab exercises a direct RDP connection to demonstrate the protocol. Port 3389 must be open in Ubuntu's firewall. In a production environment, restrict access with firewall rules (e.g. allow only specific source IPs) or route RDP through a VPN.

→ `sudo ufw allow 3389/tcp`



```
shcherba@Ubuntu-26-04:~$ sudo ufw allow 3389/tcp
Rules updated
Rules updated (v6)
```

Fig 6.4 — Port 3389 opened in the UFW firewall for RDP traffic

6.5 Configure the Desktop Environment for xrdp

xrdp reads the ~/.xsession file in the connecting user's home directory to determine which desktop environment to launch. Without this file, RDP sessions may start with no desktop or a minimal window manager.

→ `echo 'startxfce4' > ~/.xsession`

```
shcherba@Ubuntu-26-04:~$ echo 'startxfce4' > ~/.xsession
shcherba@Ubuntu-26-04:~$ cat ~/.xsession
startxfce4
```

Fig 6.5 — ~/.xsession created, directing xrdp to launch XFCE4

6.6 Connect to Ubuntu from Parrot via RDP

→ `xfreerdp /v:<UBUNTU_IP> /u:<username> /dynamic-resolution /cert:ignore`

```
$xfreerdp3 /v:192.168.56.104 /u:shcherba /dynamic-resolution /cert:ignore
```

Fig 6.6 — xfreerdp command connecting to Ubuntu via RDP

Flag	Purpose
/v:<UBUNTU_IP>	Specifies the RDP server address — Ubuntu's IP on the Host-Only network.
/u:<username>	The Ubuntu Linux user account. xrdp authenticates against the system's PAM stack using the same credentials as a local login.
/dynamic-resolution	Enables dynamic display resolution adjustment as the RDP window is resized.
/cert:ignore	Suppresses the self-signed certificate warning. xrdp generates a self-signed TLS certificate by default. In production, provision a CA-signed certificate and remove this flag.

6.7 Resulting RDP Session

After entering the Ubuntu user password at the xrdp login screen, a full XFCE4 desktop session is established over the RDP protocol.

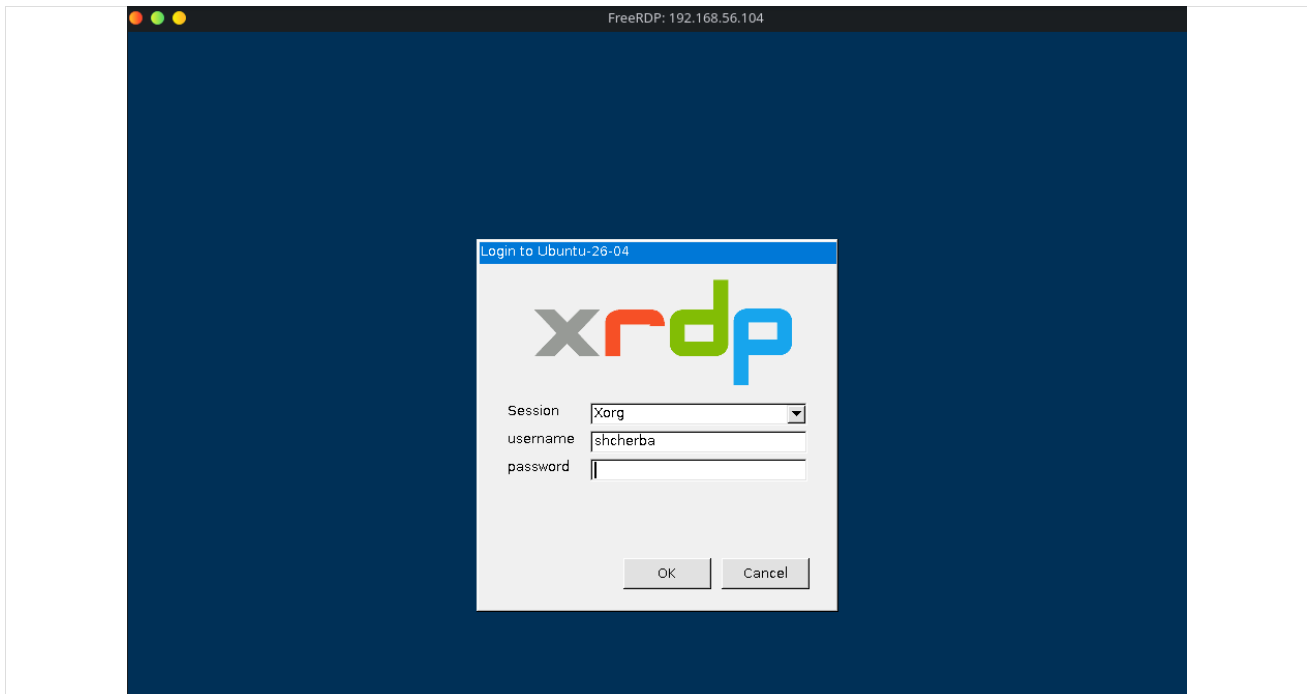


Fig 6.7 — Ubuntu XFCE4 desktop session established via RDP on Parrot

xrdp vs VNC Security Comparison

With VNC, encryption is the operator's responsibility — the SSH tunnel was added manually. With xrdp, TLS encryption is built into the RDP protocol and applies automatically. However, xrdp binds port 3389 to all network interfaces by default, making it directly reachable from the network (unlike the localhost-bound TigerVNC setup). Production hardening: enable Network Level Authentication (NLA), restrict port 3389 with firewall rules or a VPN, replace the self-signed certificate with a CA-signed one, and audit xrdp logs regularly.

The End.